

MobProg Midterm Reviewer

CS0011 | *Modules 1 To 5*

Dawn Pamesa | CS0011 Mobile Programming

Written Midterm | Wednesday, October 7, 2026 | 1 PM

Coverage: Modules 1 To 5 | Bring Your Own Device

Source And Reading Guide

Built only from the five module slide-text files and image_slides_notes.md in MobProg Exam Prep/sources. Version dates and comparison wording are presented as course material. Blue Memory Aid boxes are study mnemonics created for this reviewer, not slide content. Red Watch Out boxes identify traps and source conflicts.

Code follows the supplied transcriptions. Extracts with ellipses are recognition fragments, not complete programs. Java and Kotlin appear together only where both are supplied. Page references such as M3 p23 refer to the source slides.

What Is Inside

Read each explanation before memorizing its table. Use the code section for syntax recognition and the final page for recall.

1. Module 1: Introduction To Android. Origins, versions, features, and architecture.
2. Module 2: Android Studio. Setup, project creation, interface, and running an app.
3. Module 3: Kotlin. Variables, control flow, classes, Java comparison, and conversion.
4. Module 4: Application Components And Resources. Building blocks, resource folders, and resource IDs.
5. Module 5: Activities. Screens, lifecycle callbacks, manifest, and layouts.
6. Code You Must Recognize. Slide code and Java/Kotlin comparisons.
7. One-Page Cram Sheet. Final recall before the exam.

Fast Memory Map

Module	Big Ideas	Must-Memorize Items
M1	Android is a Linux-based mobile OS; its stack supports apps.	2005 purchase; version/codename/year; five architecture layers.
M2	Create, edit, then run a project.	JDK + Studio; six IDE parts; USB debugging; Device Manager.
M3	Kotlin syntax and object construction.	val/var; lateinit; ?/!!; open; init order; Ctrl+Alt+Shift+K.
M4	Components do jobs; resources hold external content.	Four main + six additional components; res/ and R; XML @ references.
M5	An Activity moves through lifecycle callbacks.	Seven callbacks; pause vs stop; manifest declaration; setContentView.

Part 1: M1 Introduction To Android

Android is an open and free mobile operating system based on Linux. Android Inc. originally developed it; Google purchased Android Inc. in 2005. The first commercial Android device was the HTC Dream.

Advantages And Features

Group	Slide Content
Advantages	Open Source; Larger Developer and Community Reach; Increased Marketing; Inter App Integration; Reduced Cost of Development; Higher Success Ratio; Rich Development Environment.
Features	Storage/database; multimedia; connectivity/wireless; messaging; apps; multi-touch.
Market-Share Pictures	July 2024 to July 2025 charts: Android 15/14 lead; iOS 18 leads. No precise percentages are supplied.

Version Table: Early Android

Years and features below follow M1 pp8-17 and the image notes. Learn the code name together with its feature hook.

Version	Codename	Year	Main Features In The Slides
1.0	Unnamed (alpha)	2008	Maps, camera, Gmail/Contacts/Google sync, browser, WiFi, Bluetooth.
1.1	Unnamed (beta)	2009	Save message attachments; business reviews/details in Maps.
1.5	Cupcake	2009	YouTube/Picasa uploads; MPEG-4/video recording; browser copy/paste.
1.6	Donut	2009	Large screens; gallery/camera; faster system apps.
2.0-2.1	Eclair	2009	Updated UI; live wallpaper; Bluetooth 2.1; Maps; minor API changes.
2.2	Froyo	2010	Animated GIF; WiFi hotspot; speed; browser uploads; numeric/alphanumeric passwords.
2.3	Gingerbread	2010	Copy/paste; UI; social networking; easier keyboard. Nexus S with Samsung.
3.0-3.2	Honeycomb	2011	Gmail; 3D UI; SD media sync; eBooks; Talk video; Flash; WiFi; Chinese handwriting.
4.0	Ice Cream Sandwich	2011	Text/spelling; WiFi Direct; photo decor; keyboard; Face Lock; camera/video; 16 browser tabs.
4.1-4.3	Jelly Bean	2012	Google Now; voice search/typing; smooth UI; camera/security; tablet accounts; 4K; Bluetooth LE; languages; USB audio; lock screen; alerts; emoji.
4.4	KitKat	2013	Screen recording; OK Google; GPS; offline music; Maps/alarm UI; keyboard emoji.

Memory Aid

Early dessert chunks: C-D-E / F-G-H / I-J-K. Sentence: Cats Dance Early; Foxes Gather Honey; Iguanas Juggle Kites. These stand for Cupcake through KitKat in version order.

Android Versions: Lollipop To Cinnamon Bun

Later slides pair numbered releases with named features. Keep the internal names for Android 10 onward separate from the earlier public dessert labels.

Version	Codename	Year	Main Features In The Slides
5.0-5.1	Lollipop	2014	ART (Android RunTime); battery/UI improvements; material design; fixes; multiple SIMs; HD voice.
6.0	Marshmallow	2015	Fingerprint; USB Type C; sleep-mode battery saving; permission requests; emoji.
7.0	Nougat	2016	Split-screen; data saver; multitasking/multi-window; storage manager; touch improvements.
8.0	Oreo	2017	Picture-in-Picture; multi-display; Google Play support; adaptive icons; notifications.
9.0	Pie	2018	Screenshot button; biometric Lockdown; cutouts; adaptive battery and brightness.
10	Queen Cake; internal Quince Tart	2019	Background location/media permissions; sharing shortcuts; dynamic photo depth; dark mode.
11	Red Velvet Cake	2020	Native screen recording; mute notifications during video; touch sensitivity; notification history; auto-revoke permissions.
12	Snow Cone	2021	Scrolling screenshots; AppSearch; auto-rotate; WiFi sharing; one-handed mode; rich insertion; overview suggestions; game APIs.
13	Tiramisu	2022	Security; reading mode; digital car keys; native LE Bluetooth; Material You options; QR scanner.
14	Upside Down Cake	2023	Large fonts/scaling; notification flashes; photo/video restrictions; protected PIN; data protection; regional preferences; predictive back; Health Connect.
15	Vanilla Ice Cream	2024	Privacy Sandbox; Health Connect; file integrity; partial sharing; camera controls; dynamic performance; sensitive notifications.
16	Baklava	2025	June 10; Live Updates foundation; adaptive large-screen apps; predictive back; QPR2 API 36.1.
17	Cinnamon Bun	2026	June 16; AppFunctions; adaptive-first API 37 targets; App Bubbles, Bubble Bar, desktop PiP.

Watch Out

M1 p30 is titled Android 16 but shows the Android 17 logo. The image notes explicitly say to treat Cinnamon Bun as Android 17. The messy p6 table gives Android 13 as 2023; p26 explicitly gives August 15, 2022, used here. The Gingerbread detail slide lists 2.3 and 2.4, while the image-note version table lists 2.3. Recognize the discrepancy; this table follows the image-note version row.

Memory Aid

L-M-N-O-P = Lollipop, Marshmallow, Nougat, Oreo, Pie. Number hook: 16 = Baklava = API 36; 17 = Cinnamon Bun = API 37. These pairings come from the source chart.

Android Architecture

The Android software stack combines applications, framework services, runtime, platform libraries, and the Linux kernel. The slides describe the kernel as providing operating-system functions and the Dalvik Virtual Machine as running mobile applications.

Layer, Top To Bottom	Components In The Diagram
Applications	Home, Contacts, Camera, SMS, Alarm, Time, Calendar, Music, Gallery, Phone, Clock, Email.
Application Framework	Activity Manager, Package Manager, NFC Service, Location Service, Windows Manager, Notification Manager, Content Providers, View System.
Android Runtime	Dalvik Virtual Machine, Zygote, Android Debug Bridge, Core Libraries.
Platform Libraries	Media Framework, OpenGL, Graphics, SGL, SSL, SQLite, Surface Manager, FreeType.
Linux Kernel	Display, Wi-Fi, Audio, Bluetooth, Camera, USB, Binder IPC, and Memory Drivers.

Memory Aid

Top-down sentence: Apps Find Runtime Libraries Below. Apps = Applications; Find = Framework; Runtime = Android Runtime; Libraries = Platform Libraries; Below = Linux Kernel. Number hook: 5 layers; the bottom layer is drivers.

Watch Out

Answer architecture component placement from the supplied M1 diagram. Its runtime list includes Dalvik, while the Lollipop version slide also names ART. Preserve the distinction between the diagram and the version feature.

Part 2: M2 Android Studio

Android Studio is where the project is created, code and layouts are edited, and the app is run. The prerequisites named in the slides are JDK and Android Studio. The system-requirement screenshot has no legible numeric requirements.

Create A Project

1. Choose Start a new Android Studio project, or File > New > New Project.
2. Select Empty Activity in the written steps, then Next.
3. Enter Name: My First App; Package name: com.example.myfirstapp. Check Use AndroidX artifacts. Change the location if needed; leave other options as directed.
4. Select Java if writing Java; the project form also offers Kotlin. Click Finish.

Watch Out

The written M2 steps say Empty Activity, but the screenshot selects Empty Views Activity. M5 also shows Empty Views Activity. Recognize the screenshot label instead of treating the two labels as identical. Do not invent RAM, disk, or OS requirements from an unreadable image.

Project Form	What The Source Shows
Fields	Name, Package name, Save location, Language, Minimum SDK.
Language	Java or Kotlin.
Minimum SDK	M2: list begins at API 16 / Android 4.1 Jelly Bean; API 24 / Android 7.0 Nougat selected. M5 example selects API 34.
Templates	No Activity; Empty Activity; Basic Views Activity; Bottom Navigation Views Activity; Empty Views Activity; Navigation Drawer Views Activity; Game; C++.

Six Interface Parts

Part	Purpose
Toolbar	Run the app and launch Android tools.
Navigation Bar	Navigate the project and open files; compact Project-window structure.
Editor Window	Create/modify code; a layout file opens the Layout Editor.
Tool Window Bar	Outer buttons expand/collapse individual tool windows.
Tool Windows	Project management, search, version control, and other tasks.
Status Bar	Project/IDE status, warnings, and messages.

Memory Aid

Six-part sentence: Tools Navigate Editors; Bars Toggle Status. Map to Toolbar, Navigation bar, Editor, Tool window bar, Tool windows, Status bar.

Run The Application

Target	Steps In The Slides
Real Device	Connect the Android phone to the computer. Enable Developer options, then USB Debugging.
Emulator	Open Device Manager > Create Device. Choose screen size and phone type, then Android SDK version.

Memory Aid

Run in two chunks: Phone = connect + debugging. Emulator = device + screen + SDK.

Part 3: M3 Kotlin

Kotlin is a concise, safe language released by JetBrains in 2016. It is compatible with Java and works on Windows, Mac, Linux, Raspberry Pi, and other platforms. The slides describe it as free, easy to learn for Java users, and supported by a large community.

Topic	Meaning Or Syntax
Uses	Android/mobile apps; web development; server-side applications; data science.
Functions	fun declares a function. The standalone example begins with main(): fun main() { println("Hello World") }
Punctuation	Statements do not need a final semicolon. // is a single-line comment; /* ... */ is a multiline comment.
val	The reference cannot be changed after initialization: val immutable = 12.
var	The reference can change: var mutable = "Hello World"; later mutable = "Hi there".
Type Inference	The variable type need not be written when Kotlin can infer it.
lateinit	lateinit var str1: String is assigned later: str1 = "late init".
Nullable Type	var str2: String? = null explicitly permits null.
Conversion Example	The converted code uses textView!!.text = Integer.toString(count). The !! asserts a non-null value; ? in TextView? permits null.

Watch Out

val fixes the reference; var permits reassignment. Do not read lateinit as an initialized value: the slide assigns str1 before printing it. The nullable str2 example deliberately starts at null. Do not confuse a nullable type (?) with the conversion example's non-null assertion (!!).

Memory Aid

Variable chunks: val = value reference locked; var = variable reference. Read the symbols aloud: ? permits null; !! asserts not null. These are recognition aids for the supplied examples.

Control Flow

Form	Source Example Or Result
if / if-else / else-if	Choose among conditions. if-else can also supply an expression value.
if Expression	val time = 20; val greeting = if (time < 18) "Good day." else "Good evening." gives Good evening.
when	Map day 1 through 7 to Monday through Sunday; else gives "Invalid day.". day = 4 gives Thursday.
Loops	while; do-while; for-loop. Array example visits 1, 5, 10, 15, 20. Range example: for (nums in 5..15).

Kotlin Classes And Construction

A class groups properties and functions. The Car examples create an object with Car(), assign its properties, then show a primary constructor that receives those properties directly. The Person example shows initialization blocks executing in their written order.

Concept	Slide Example And Meaning
Class And Object	<code>class Car { var brand = ""; var model = ""; var year = 0 }; val c1 = Car(). Set Ford, Mustang, 1969.</code>
Primary Constructor	<code>class Car(var brand: String, var model: String, var year: Int); Car("Ford", "Mustang", 1969).</code>
Functions	<code>drive()</code> prints "Wrooom!"; <code>speed(maxSpeed: Int)</code> prints "Max speed is: " plus <code>maxSpeed</code> . Calls: <code>c1.drive()</code> ; <code>c1.speed(200)</code> .
init Blocks	<code>Person("Abe")</code> prints first, second, third init block, then <code>Name = Abe</code> . The last block assigns <code>this.name = _name</code> .
Default Arguments	<code>class employee(emp_id: Int = 100, emp_name: String = "abc")</code> . <code>employee(18018, "Sagnik")</code> supplies both; <code>employee(11011)</code> uses <code>abc</code> ; <code>employee()</code> uses <code>100</code> and <code>abc</code> .
Secondary Constructors	Add defines <code>constructor(a, b)</code> , <code>constructor(a, b, c)</code> , and <code>constructor(a, b, c, d)</code> , all <code>Int</code> parameters. <code>Add(5, 6)</code> prints "Sum of 5, 6 = 11".
Inheritance	<code>open class MyParentClass</code> has <code>val x = 5</code> . <code>MyChildClass</code> : <code>MyParentClass()</code> inherits it and prints <code>x</code> in <code>myFunction()</code> .

Watch Out

Kotlin classes are final by default in the image notes: the parent needs open for inheritance. init blocks run in order of appearance, not in an arbitrary order. A constructor default is used when that argument is omitted.

Memory Aid

Construction chunks: Declare, Create, Initialize, Call. Number hook for Person: first + second + third + name. Inheritance sentence: Open parents let children enter.

Kotlin Versus Java: Slide Comparison

Feature	Kotlin	Java
Extension Functions	Already available	Create a class
Null Safety	Available	Not available in slide comparison
Static Members	No static member for a class	Available
String Templates	Two string-literal types; expressions	Slide says available, without Kotlin-like expressions
Wildcard Types	Not available	Available
Smartcasts	Available	Not available
Checked Exceptions	Row heading: No Checked Exceptions	Slide describes these as problematic
Operator Overloading	Users provide a way to invoke functions	Operators tied to particular Java types
Constructors	Primary and secondary	Parameters initialize attributes

Feature	Kotlin	Java
Type System	Nullability, inference, universal guards (slide wording)	Reference types related to classes

Watch Out

M3 p21 labels its row "No Checked Exceptions" but says "Kotlin removed exceptions entirely" in the cell. These are different claims. Memorize the row heading; the broader cell wording is an unresolved source inconsistency, not a rule established by these notes.

Java-To-Kotlin Conversion

Android Studio supports Java and Kotlin in the same project. The slide diagram shows both compiling to JVM bytecode. The conversion slides give two workflows: convert a whole file or paste Java into a Kotlin file.

Method 1: Convert A Whole Class Or File

1. Open the Java source file, such as MainActivity.java.
2. In Android Project view, right-click the file and choose Convert Java File to Kotlin File, or use Ctrl+Alt+Shift+K while the file is open.
3. If prompted, permit Kotlin configuration. Select All modules and the latest installed Kotlin compiler, then OK. Android Studio changes the app module build.gradle.
4. Choose Convert Java File to Kotlin File again. The file extension changes from .java to .kt.

Method 2: Add A Kotlin File And Paste Java

1. Create a Kotlin Class/File inside the project's java folder and give it a name.
2. Use Configure on the "Kotlin is not configured" alert. Select All modules and the latest installed compiler; click OK and let build.gradle changes finish.
3. After the project builds, open the Kotlin file and paste the Java code. Android Studio converts it automatically.

Watch Out

The shortcut has all three modifiers: Ctrl + Alt + Shift + K. The screenshot shows compiler 1.4.30-RC, but the written instruction is to choose the latest compiler installed on the computer; the screenshot value is not a universal requirement.

Memory Aid

Whole-file route: Open, Convert, Configure, Convert. Paste route: Create, Configure, Copy. Shortcut hook: three modifiers plus K for Kotlin.

Part 4: M4 Application Components And Resources

Application components are the building blocks of an Android app. AndroidManifest.xml describes the components and how they interact. The four main components divide screen interaction, background work, communication, and data handling.

Main Component	Job And Example
Activities	A screen with a UI; handles user interaction. Email can have separate list, compose, and read activities. One is marked for launch.

Main Component	Job And Example
Services	Long-running background work, such as music during another app or a network fetch without blocking activity interaction.
Broadcast Receivers	Respond to broadcasts from apps or the system, such as downloaded data becoming available. Messages are Intent objects.
Content Providers	Supply data to other apps on request. ContentResolver methods handle requests. Data may be in a file system, database, or elsewhere; the provider implements transaction APIs.

Six Additional Components

Component	Description
Fragments	A portion of the UI inside an Activity.
Views	On-screen UI elements, including buttons, lists, and forms.
Layouts	View hierarchies controlling screen format and appearance.
Intents	Messages wiring components together.
Resources	External elements, including strings, constants, and drawable pictures.
Manifest	Application configuration file.

Memory Aid

Four jobs: Screen, Background, Broadcast, Data = Activity, Service, Receiver, Provider. Six extras: Friendly Views Lay Into Resource Maps = Fragments, Views, Layouts, Intents, Resources, Manifest. Number hook: 4 main + 6 additional.

Watch Out

A Fragment is a portion of an Activity UI. A View is a UI element, while a Layout organizes a view hierarchy. ContentProvider supplies data; ContentResolver handles the requests described in the slides.

Resource Organization And Access

Resources are additional files and static content kept separately from code under res/. Compiling generates an R class containing resource IDs. Code uses resource type and name; XML examples use @type/name.

Directory	Stored Content	Access In The Sources
animator/	Property-animation XML; preferred for property animations.	R.animator (M4 p18 naming pattern)
anim/	Tween-animation XML; can also hold property animations.	R.anim
color/	Color state-list XML.	R.color
drawable/	PNG, .9.png, JPG, GIF; bitmaps, nine-patches, state lists, shapes, animation/other drawables.	R.drawable
layout/	UI-layout XML.	R.layout

Directory	Stored Content	Access In The Sources
menu/	Options, context, and submenu XML.	R.menu
raw/	Arbitrary files in raw form.	Resources.openRawResource(R.raw.filename)
values/	Strings, integers, colors, arrays, dimensions, styles.	Each child element defines a resource: <string> -> R.string; <color> -> R.color.
xml/	Arbitrary XML read at runtime; search configuration.	R.xml (M4 p18 pattern); Resources.getXML() as written in p17.
font/	TTF, OTF, TTC, or XML with <font-family>.	R.font (M4 p18 naming pattern)
mipmap/	icon.png in the example project tree.	R.mipmap.icon (M4 p18 naming pattern)

Watch Out

M4 p13 repeats conflicting descriptions under anim/. Its distinguishing statement prefers animator/ for property animation and describes anim/ for tween animation. Rows marked naming pattern apply p18's general R/subdirectory/name rule; those exact members are not separately printed in the transcription.

Values Files And Assets

Item	Rule
values/ Naming	A values XML file can define multiple resources under <resources>. Each child defines one resource; the filename can vary.
Conventions	arrays.xml: typed arrays; colors.xml: colors; dims.xml: dimensions; strings.xml: strings; styles.xml: styles.
Other Resource XML	Other res/ subdirectories define a single resource based on the XML filename, as described in the slides.
assets/ Versus raw/	Use assets/ when original filenames and hierarchy are needed. Assets receive no resource ID; read them using AssetManager.

Memory Aid

Resource access chunks: Code = R.type.name; XML = @type/name. Values = many child resources; raw = resource ID; assets = AssetManager.

Part 5: M5 Activities

An Activity represents one screen of an app, similar to a desktop application window. An app can contain one or more activities, starts with its main activity, and may open additional activities. Activity classes are subclasses of android.app.Activity in the module explanation.

Lifecycle Callbacks

Callback	When It Is Called
onCreate()	First callback when the activity is first created.

Callback	When It Is Called
onStart()	When the activity becomes visible to the user.
onResume()	When the user starts interacting with the application.
onPause()	When the current activity is paused as another comes to the foreground; the paused activity does not receive user input.
onStop()	When the activity is no longer visible.
onDestroy()	Before the activity is destroyed by the system, as described in the callback table.
onRestart()	When the activity restarts after being stopped.

Read The Lifecycle Diagram

Situation	Path In M5 p7
Launch	onCreate -> onStart -> onResume -> running.
Another Activity In Front	onPause; if the user returns, onResume.
No Longer Visible	onPause -> onStop.
Return After Stop	onRestart -> onStart -> onResume.
Finish/Destroy	onDestroy -> activity shut down.
Higher-Priority Apps Need Memory	Diagram permits process killing from the paused/stopped paths. Navigating back leads to onCreate.

Watch Out

onPause concerns lost interaction; onStop means no longer visible. M5 p6 also says a paused activity "cannot execute any code"; this is overbroad relative to the callback-flow diagram. Use the diagram to distinguish pause, resume, stop, and restart. Do not assume the process-killed branch passes through onDestroy: the supplied diagram shows it separately.

Memory Aid

Launch sentence: Create, Show, Respond = onCreate, onStart, onResume. Return after stop adds Restart before Start. Number hook: 7 named callbacks; pause and stop are separate.

Create And Declare The Screen

1. Create the Activity subclass. The Kotlin walkthrough uses Empty Views Activity, a name, Kotlin, a minimum SDK, and Finish.
2. Declare the Activity in AndroidManifest.xml with an <activity> child of <application>.
3. Use MainActivity.kt for Kotlin code and activity_main.xml for the main layout.
4. Load res/layout/activity_main.xml during onCreate with setContentView(R.layout.activity_main). Run with the emulator.

Watch Out

An Activity must be declared in the manifest. The Activity class and the layout XML serve different purposes: screen behavior versus screen design. Writing a class alone does not supply the required manifest declaration.

Code You Must Recognize

These examples reproduce the available slide fragments, with whitespace adjusted to fit. Java/Kotlin pairs come from slides that supply both. Where only one language is supplied, it is labeled instead of inventing a translation.

TextView And onCreate: M3 pp23-24

Declare a TextView, find its ID, then change its text. The supplied callback headers differ in syntax.

Java	Kotlin
TextView textView;	lateinit var textView: TextView
@Override public void onCreate(Bundle savedInstanceState)	override fun onCreate(savedInstanceState: Bundle?)
textView = (TextView) findViewById(R.id.text_view);	textView = findViewById(R.id.text_view)
textView.setText("New Text");	textView.text = "New Text"

Main Activity And Layout: M2 p9, M5 pp15-16

Both source examples subclass AppCompatActivity and load activity_main; these are the transcribed headers and calls, not complete class files.

Java	Kotlin
public class MainActivity extends AppCompatActivity { // ... }	class MainActivity : AppCompatActivity() // body not fully transcribed
@Override protected void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); setContentView(R.layout.activity_main); }	override fun onCreate(savedInstanceState: Bundle?) // calls shown in M5: enableEdgeToEdge() setContentView(R.layout.activity_main) ViewCompat. setOnApplyWindowInsetsListener(...)

Counter Conversion: M3 pp27-31

The converted Kotlin example keeps a nullable TextView and asserts it is non-null when assigning the count text.

Java Fragments	Kotlin Fragments
private TextView textView; private int count;	private var textView: TextView? = null private var count = 0
// buttonOnClick increments count // full body not transcribed	count++ textView!!.text = Integer.toString(count)

Watch Out

These excerpts teach recognition. Ellipses and explanatory comments identify untranscribed bodies; they are not instructions to paste an incomplete class into a project. Keep the nullable `TextView?` and `!!` assertion distinct.

Component Subclasses And Resource Code

M4 gives the following Java skeletons only. They identify the superclass for each component; the receiver and provider method signatures are incomplete as transcribed.

Four Component Skeletons: M4 pp6-9

Recognize extends plus the component base class.

Component	Java Slide Fragment
Activity	<code>public class MainActivity extends Activity { }</code>
Service	<code>public class MyService extends Service { }</code>
Broadcast Receiver	<code>public class MyReceiver extends BroadcastReceiver { public void onReceive(context, intent){} }</code>
Content Provider	<code>public class MyContentProvider extends ContentProvider { public void onCreate(){ } }</code>

Watch Out

M4 skeletons omit implementation details, including parameter types in `onReceive`. Treat them as slide recognition fragments, not complete compilable component implementations. The sources provide no Kotlin counterparts for these four skeletons.

Resources In Java: M4 pp19-22

Use `R.id` to find a view, `R.drawable` for the picture, `R.string` for the string, and `R.layout` for the layout.

Purpose	Java Slide Code
Set An Image	<code>ImageView imageView = (ImageView) findViewById(R.id.myimageview); imageView.setImageResource(R.drawable.myimage);</code>
Set Resource Text	<code>TextView msgTextView = (TextView) findViewById(R.id.msg); msgTextView.setText(R.string.hello);</code>
Load A Layout	<code>public void onCreate(Bundle savedInstanceState) { super.onCreate(savedInstanceState); setContentView(R.layout.activity_main); }</code>

Memory Aid

Read each ID in three chunks: R / category / name. imageView uses drawable.myimage; msgTextView uses string.hello; the activity uses layout.activity_main.

XML Resources, Manifest, And Layouts

XML examples connect named resources to view attributes and register an Activity. XML is shown separately because these are shared XML fragments, not Java or Kotlin source files.

String And Color Resources: M4 pp20 And 23

The two slides use different hello string values; keep each definition with its example.

Source	XML Fragment
M4 p20: strings.xml	<string name="hello">Hello, World!</string>
M4 p23: strings.xml	<color name="opaque_red">#f00</color> <string name="hello">Hello!</string>
M4 p23: EditText Attributes	android:textColor="@color/opaque_red" android:text="@string/hello"

Manifest Declaration: M5 p10

Place the Activity entry inside the application element.

XML Slide Fragment
<manifest> <application> <activity android:name=".ExampleActivity" /> </application> </manifest>

Layout Pictures To Recognize

Source	Container And Children
M4 p21	activity_main.xml: LinearLayout with fill_parent and vertical orientation. TextView: @+id/text, "Hello, I am a TextView". Button: @+id/button, "Hello, I am a Button". Children use wrap_content.
M5 pp17-18	activity_main.xml: androidx.constraintlayout.widget.ConstraintLayout with match_parent. TextView says "Hello Android!" and is constrained Bottom/End/Start/Top to parent.
M4 p12 Project Tree	src/MyActivity.java; res/drawable/graphic.png; res/layout/main.xml and info.xml; res/mipmap/icon.png; res/values/strings.xml.

Watch Out

R.string.hello is a code resource reference; @string/hello is the XML form. The layout pictures also show @+id/text and @+id/button. Preserve the plus sign when recognizing those source attributes.

Kotlin Syntax Fragments

These M3 examples are Kotlin-only in the supplied sources. Read the value or output before moving to the next row.

Variables And Branches: M3 pp9-12

Recognize reassignment, deferred initialization, nullable types, and expression results.

Kotlin	Read It As
<pre>var mutable = "Hello World" val immutable = 12 mutable = "Hi there"</pre>	mutable changes; immutable keeps its reference.
<pre>lateinit var str1: String var str2: String? = null str1 = "late init"</pre>	str1 is assigned later; str2 allows null.
<pre>val time = 20 val greeting = if (time < 18) "Good day." else "Good evening."</pre>	greeting becomes Good evening.
<pre>val result = when (day) { 1 -> "Monday" // ... through 7 -> "Sunday" else -> "Invalid day." }</pre>	day = 4 gives Thursday in the full slide mapping; intervening cases are omitted here.

Loops And Inheritance: M3 pp13 And 20

The loop visits the array entries; the child inherits x from the open parent.

Kotlin	Read It As
<pre>val nums = arrayOf(1, 5, 10, 15, 20) for (x in nums) { println(x) } // range header from slide: for (nums in 5..15)</pre>	Array output: 1, 5, 10, 15, 20. A second example uses a range.
<pre>open class MyParentClass { val x = 5 } class MyChildClass: MyParentClass() { fun myFunction() { println(x) } }</pre>	Calling the child's myFunction prints 5.

Memory Aid

Trace in chunks: declaration, assigned value, condition, selected result. For inheritance: open parent, child constructor call, inherited property.

One-Page Cram Sheet

CS0011 | Wednesday, October 7, 1 PM | Modules 1 To 5 | Bring Your Own Device

Recall	Answer
M1 Origins	Linux; Android Inc.; Google bought it in 2005; open/free; HTC Dream.
Version Chunks	C-D-E: 2009; F-G: 2010; H-I: 2011; J: 2012; K: 2013; L: 2014; M: 2015; N: 2016; O: 2017; P: 2018.
10 To 17	10 Quince Tart 2019; 11 Red Velvet Cake 2020; 12 Snow Cone 2021; 13 Tiramisu 2022; 14 Upside Down Cake 2023; 15 Vanilla Ice Cream 2024; 16 Baklava 2025; 17 Cinnamon Bun 2026.
Stack	Applications > Framework > Runtime > Platform Libraries > Linux Kernel.
M2 Setup/Run	JDK + Studio. Phone: USB debugging. Emulator: Device Manager > Create Device > screen/phone > SDK.
IDE Six	Toolbar; navigation bar; editor; tool window bar; tool windows; status bar.
M3 Syntax Traps	val fixed reference; var reassignable; lateinit assigned later; ? nullable; !! non-null assertion; open permits inheritance; init blocks run in order.
Convert	Ctrl+Alt+Shift+K. Whole file: open/convert/configure/convert. Paste: create/configure/copy. .java -> .kt.
M4 Four + Six	Activity, Service, BroadcastReceiver, ContentProvider. Fragments, Views, Layouts, Intents, Resources, Manifest.
Resource Access	Code R.type.name; XML @type/name. values/ uses child elements. raw/ gets an ID; assets/ uses AssetManager.
M5 Lifecycle	Create > Start > Resume; Pause > Resume on return; Stop > Restart > Start. Stop = no longer visible. Process-killed path returns to Create.
Hands-On	Declare <activity> under <application>. onCreate loads setContentView(R.layout.activity_main). findViewById finds the view. Java setText(...); Kotlin .text = ...

Watch Out

Cinnamon Bun is 17 despite the slide title. Android 13: use the detailed 2022 slide. Pause is not stop. Slide skeletons and ellipses are not complete programs.